To read a word from I/O
↳ memory mapped I/O

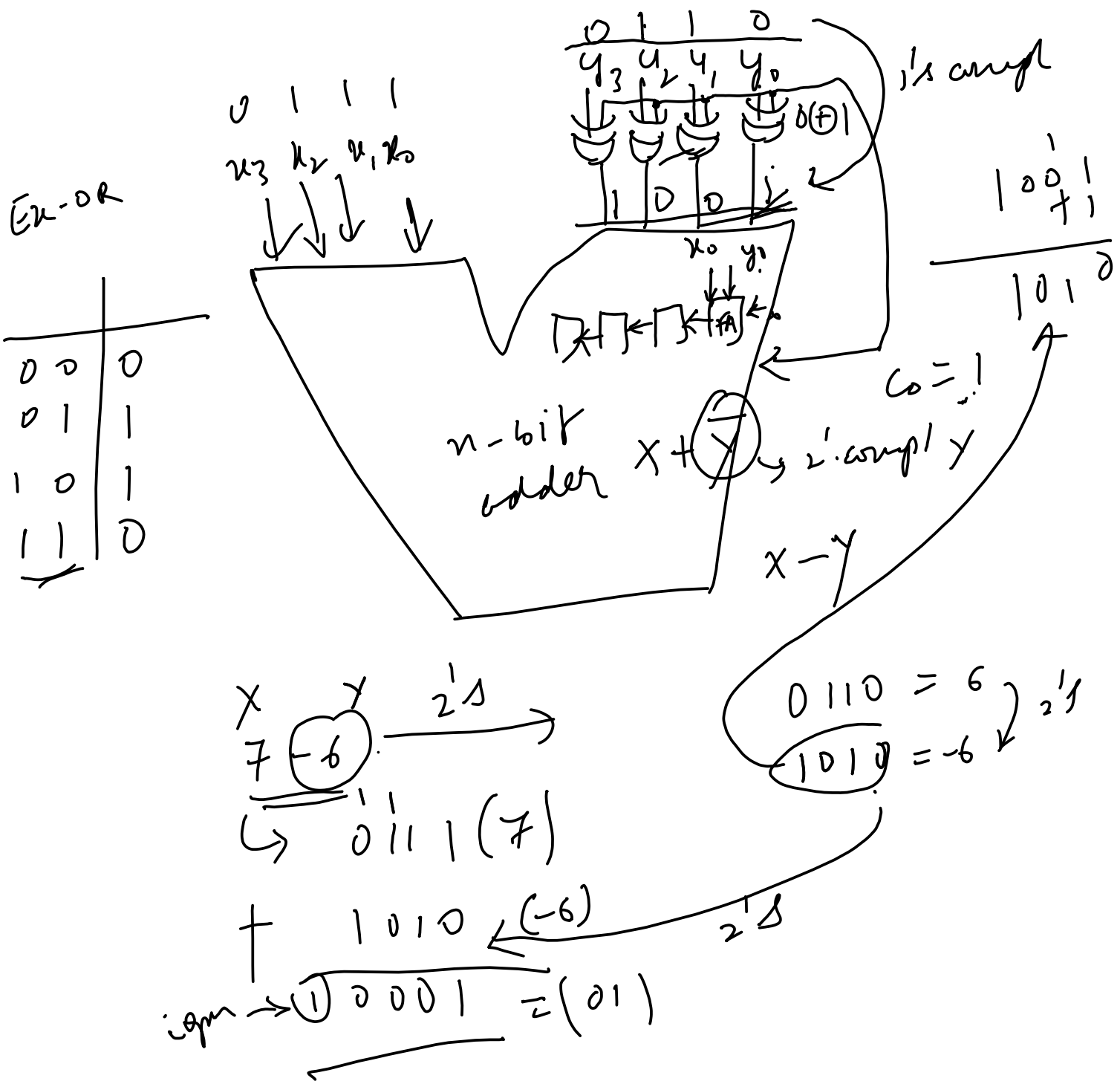
→ Load R2, DATAIN
→ Store R3, DATAOUT

IN ↳ key
OUT ↳ printer

Status of I/O → Ready
Busy

↳ polling
↳ more M.M access
↳ Interrupts -
↳ saving and restoring of register contents
↳ overhead

Block of data


Ex-OR
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 0
0 1 1 1
x3 x2 x1 x0
0 1 1 0
y3 y2 y1 y0
0 ⊕ 1
1 0 0
x0 y1
FA
1's compl
1 0 0 1
+ 1
1 0 1 0
Co = 1
n-bit adder
X + Y
2's compl Y
X − Y
X Y 2's
7 -6
0 1 1 1 (7)
+ 1 0 1 0 (-6)
2's
1 0 0 0 1 = (01)
carry
0 1 1 0 = 6
1 0 1 0 = -6
2's

CLA

$$S_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

$$= x_i y_i + (x_i + y_i) c_i$$

$$c_{i+1} = G_i + P_i c_i$$

$$\boxed{c_1 = G_0 + P_0 c_0}$$

$$c_2 = G_1 + P_1 c_1 = G_1 + P_1 (G_0 + P_0 c_0)$$

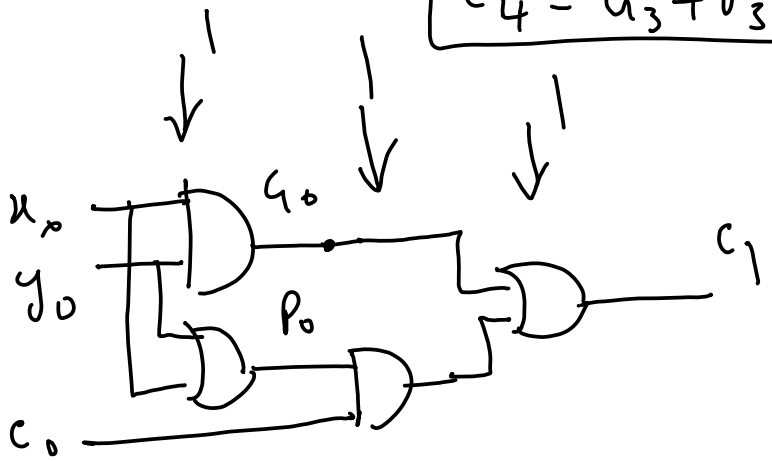
$$\boxed{c_2 = G_1 + P_1 G_0 + P_1 P_0 c_0}$$

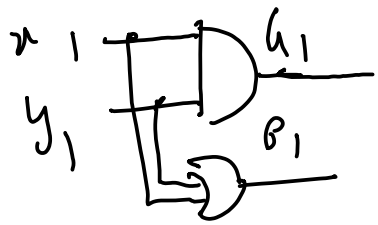
$$c_3 = G_2 + P_2 c_2 = G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 c_0)$$

$$\boxed{c_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0}$$

$$c_4 = G_3 + P_3 c_3 = G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0)$$

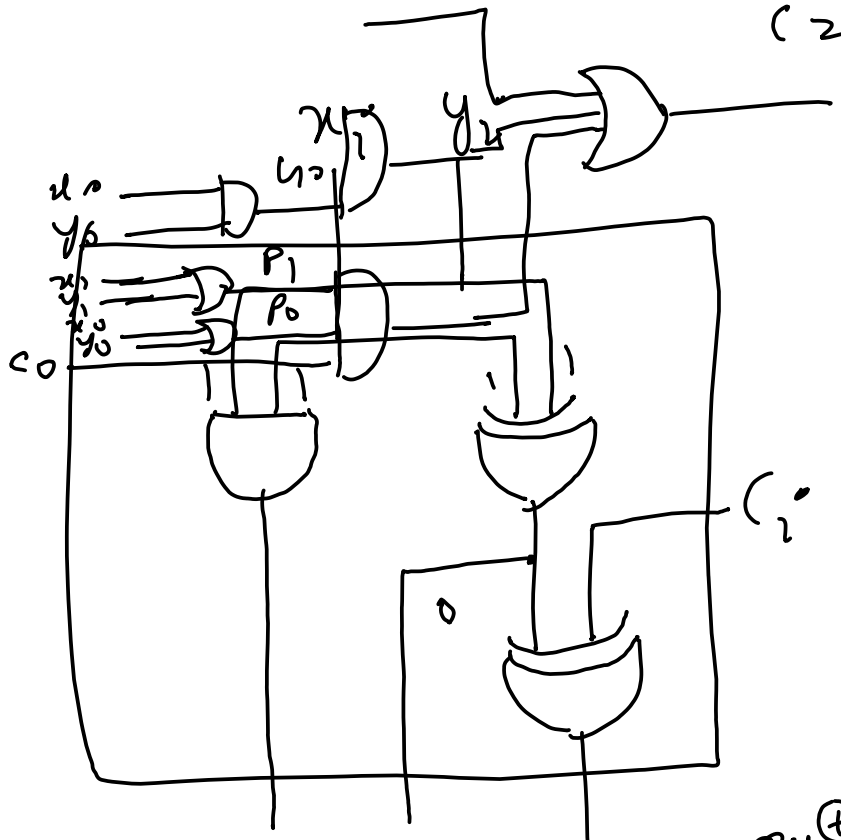
$$\boxed{c_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0}$$





3 gate delays

– c_1, c_2, c_3, c_4



$G_i = x_i y_i = 1 \cdot 1 = 1$

$P_i = x_i + y_i$

$\Rightarrow x_i \oplus y_i$

x	y	+	$\oplus$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

$\oplus C_i$

$x_i \cdot y_i = G_i$ $\quad P_i = x_i \oplus y_i$ $\quad S_i = x_i \oplus y_i \oplus C_i$

$C_{i+1} = G_i + P_i \cdot C_i$

$\oplus$

$1 \oplus = 1 + 0 \, C_i$

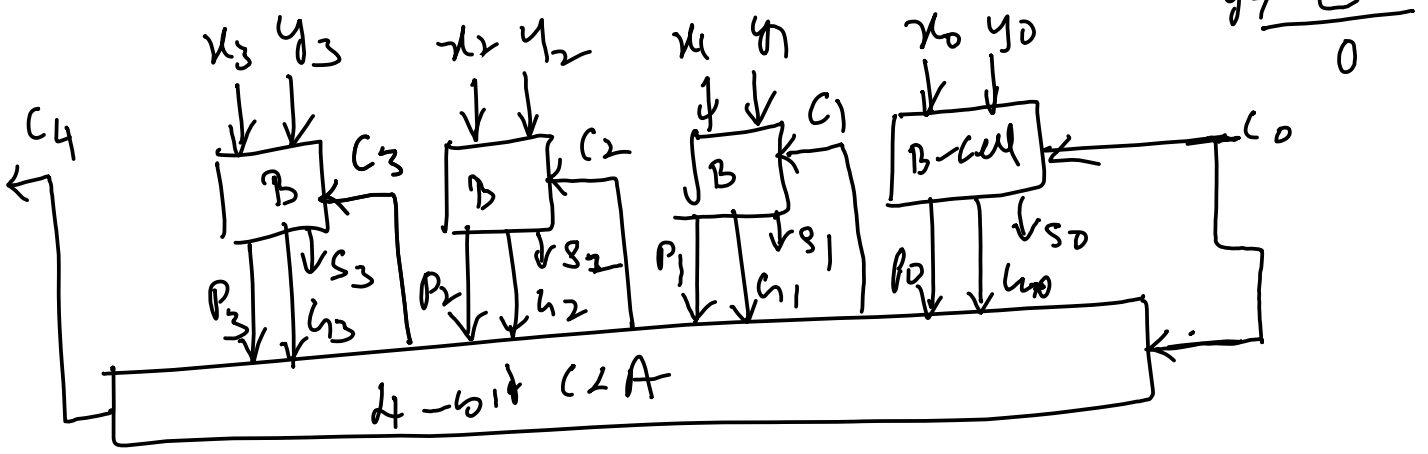
$x_i = y_i = 1$

$G_i = 1$ hence $C_{i+1} = 1$ even if $P_i = 0$

$x \to 1\ 1\ 0\ 1$

$y \to 1\ 1\ 0\ 1$

0



	S_0	$C_1 - C_4$	$S_1 - S_4$
gate delays →	1	3	4

Cascaded CLA

32-bit adder — 4 bit CLA

C_{32} = ?

$\frac{32}{4} = 8$ blocks

$= 7 + 1$ (1 → 3 gate delays)

$7 \times 2 + 3$ gate delays

$14 + 3 = 17$

$S_0 - S_{31}$ $\quad C_{32}, C_{31}, C_{30}, C_{29}$

S_{31} = ? $\quad C_{31} = 17$

$S_{31} = 17 + 1 = 18$ gate delays

C_{32} = gate delays — Ripple carry adder

$32 \times 2 = 64$ gate delays

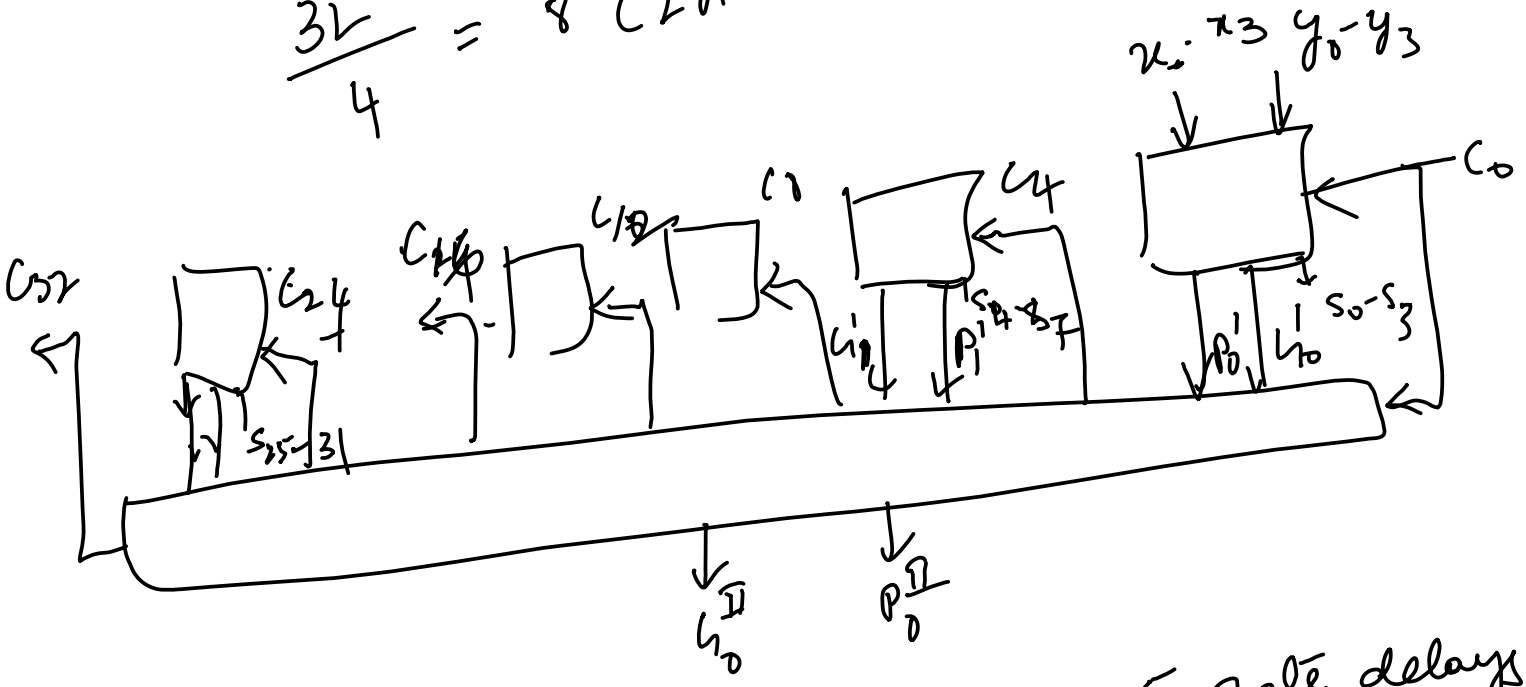
S_{31} = ? $\quad C_{31} \Rightarrow 31 \times 2 = 62$ gate delays

$S_{31} = 62 + 1 = 63$ gate delays

Blocked CLA

32 bit addition using 4 bit CLA

$$\frac{32}{4} = 8 \text{ CLA}$$



$C_{32}, C_{24}, C_{16}, C_{12}, C_8, C_4$ — 5 gate delays

$C_{31} = 5 + 2 = 7$

$S_{31} = 8$ gate delays

Multiplication — unsigned

$13 \times 11 = 143$

(M) 1101 × 1011 (Q)

1101 (13)
1011 (11)

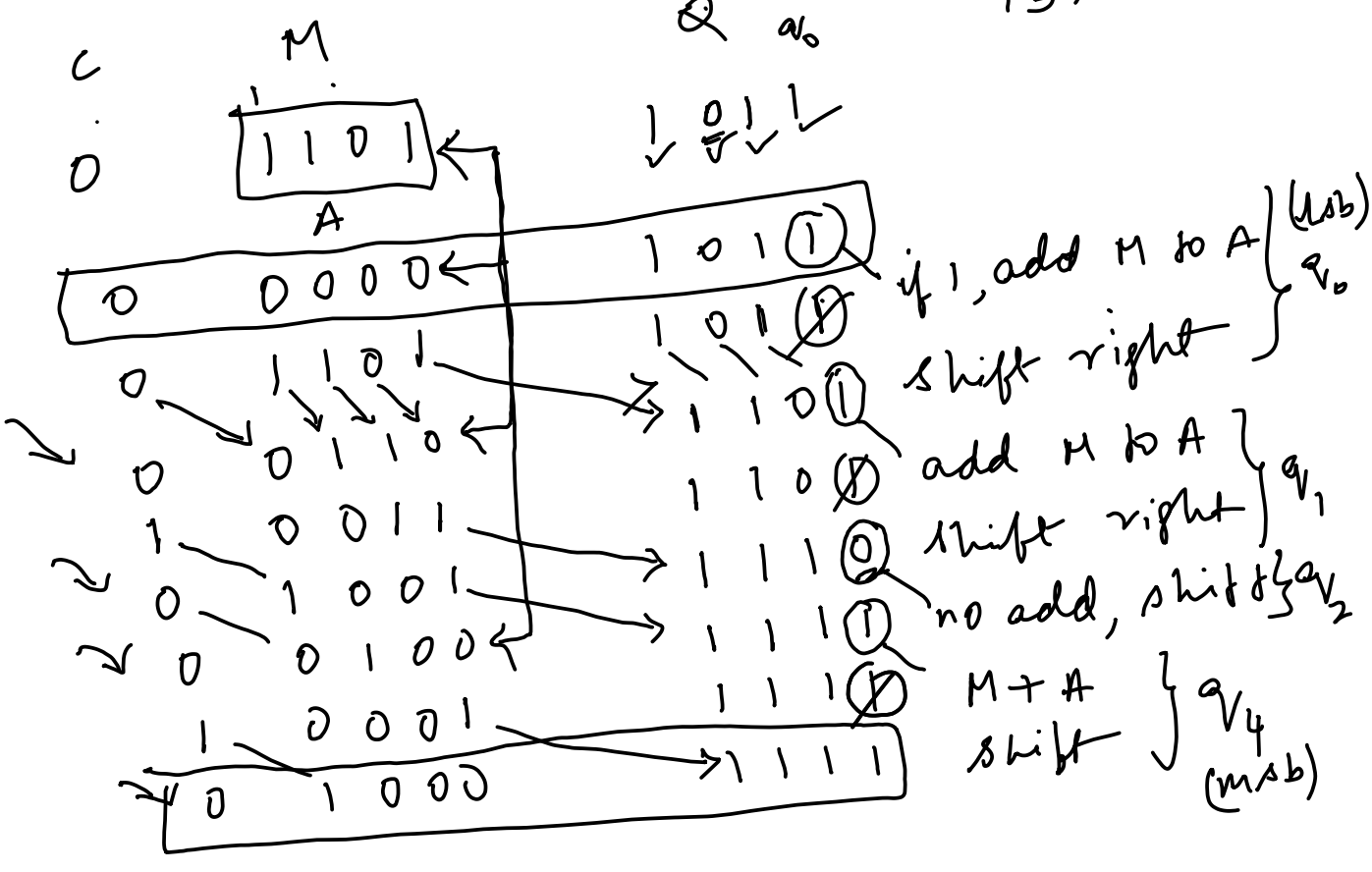
1 1 0 1
1 1 0 1 ×
0 0 0 0 × ×
1 1 0 1 × × ×
1 0 0 0 1 1 1 1 = 143

partial products

128 8 4 2 1

Multiplication of unsigned nos. using sequential

$13 \times 11 = 143$



Signed multiplication

Eg: -13×11

-13 → 2's

(-13) (11)

10011×01011

sign

8 4 2 1

0 1 1 0 1 (13) → 2's → 1 0 0 1 1 (−13)

0 1 0 1 1 (11)

2 2 1 1 1 1 1

1 1 1 1 1 0 0 1 1 — pp size = 2n (sign extension)

1 1 1 1 1 0 0 1 1 x

0 0 0 0 0 0 0 0 x x

1 1 1 0 0 1 1 x x x

0 0 0 0 0 0 x

1 1 0 1 1 1 0 0 0 1 (−143) (2n)

2's → 0 0 1 0 0 0 1 1 1 1 (143)

8 4 2 1

1 0 0 (4)

carry Sum

2 1

1 0

1 0 1 (5)

1 → sum

c

Booth algorithm → multiplier has block of 1's

1 0 0 1 0 1 x 0 1 1 1 1 0

→ 4 1's

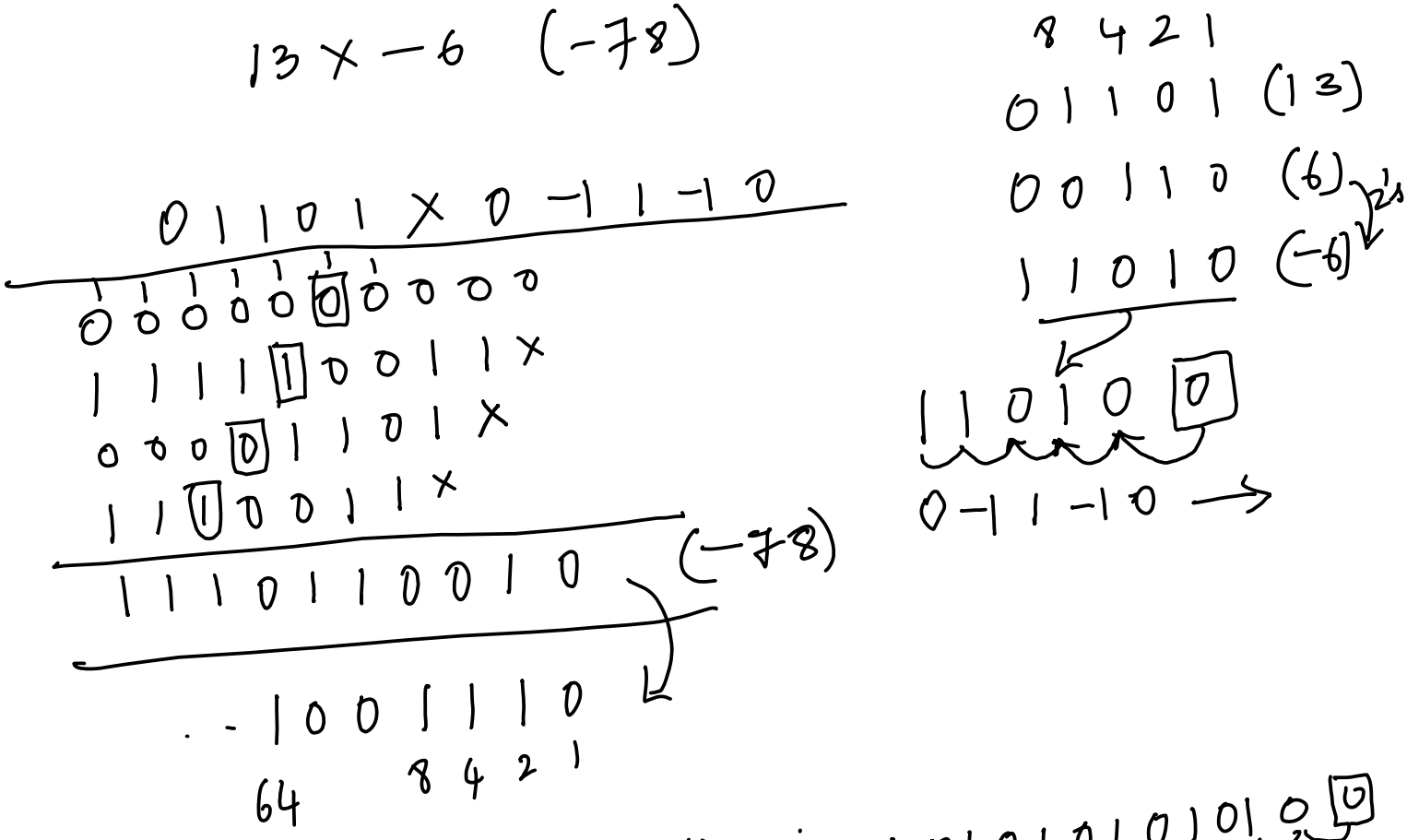
Booth recoding

i	i-1	Recoded i^{th}
0	0	0
0	1	+1
1	0	−1
1	1	0

i i-1

0 1 1 1 1 0 0 — multiplier

+1 0 0 0 −1 0 — Recoded



Eg:- worst case multiplier : 1 0 1 0 1 0 1 0 1 0 1 0

best case multiplier – 0 0 1 1 1 1 0 1 1 1 1 1 0

ordinary multiplier – 1 1 0 0 0 1 0 1 1 0 1 1 1 1 0

Multiplier		Version of multiplicand selected by bit i
Bit i	Bit $i-1$	
0	0	$0 \times M$
0	1	$+1 \times M$
1	0	$-1 \times M$
1	1	$0 \times M$

Figure 9.12 Booth multiplier recoding table.

MD MR → Recode

Eg 2: $-13 \times -20 = 260$ use Booth's algorithm

16 8 4 2 1

0 0 1 1 0 1 = 13

⇒ 1 1 0 0 1 1 = −13 ✓

0 1 0 1 0 0 = 20

1 0 1 1 0 0 = −20 ✓

1 0 1 1 0 0 0

⇒ −1 +1 0 −1 0 0

−13

1 1 0 0 1 1 × −1 1 0 −1 0 0

0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 × 2's

0 0 0 0 0 0 1 1 0 1 ×

0 0 0 0 0 0 0 0 0 0 × 2's

1 1 1 1 0 0 1 1 ×

0 0 0 1 1 0 1 ×

0 0 0 1 0 0 0 0 0 1 0 0 (260)

256 . . − 4 2 1

$n/2$ = 3 summands (Bit pair)

4 2 1

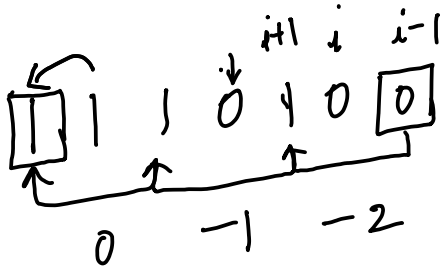
1 0 0 → 5

→ 2 − 1

Bit-pair recoding

i+1	i	i−1	Recoded i^{th} bit
0	0	0	0
0	0	1	+1
0	1	0	+1
0	1	1	+2
1	0	0	−2
1	0	1	−1
1	1	0	−1
1	1	1	0

Eg: 13 × −6



8 4 2 1

0 1 1 0 1 = 13

0 0 1 1 0 = 6

1 1 0 1 0 = −6

→ −1 × 2

0 1 1 0 1 × 0 −1 −2

1 1 1 1 1 1 0 0 1 1 0

1 1 1 1 0 0 1 1 × ×

0 0 0 0 0 0 × ×

1 1 1 0 1 1 0 0 1 0 (−78)

0 1 1 0 1 × −1

1 0 0 1 1 × 2

1 0 0 1 1 0

Eg2: −11 × 27 = −297

16 8 4 2 1

0 0 1 0 1 1 (11)

0 1 1 0 1 1 (27)

2's

(−11) ↓

1 1 0 1 0 1 × 2 −1 −1

0 1 1 0 1 1 0

+2 −1 −1

0 0 0 0 0 0 0 1 0 1 1

0 0 0 0 0 0 1 0 1 1 × ×

1 1 1 0 1 0 1 0 × ×

1 1 1 0 1 1 0 1 0 1 1 1 (−297)

1 0 0 1 0 1 0 0 1

256 32 8 1

Multiply each of the following pairs of signed 2's complement numbers using

(i) Booth algorithm

(ii) Bit-pairing of the multipliers.

In each case, assume that A is multiplicand and B is multiplier.

(a) $A = 110011$ $\quad$ $B = 101100$

-13 $\quad\times\quad$ $-20 = 260$

i) $110011 \times -110-100$ $\quad\Big|\quad B = 101100\,\boxed{0}$

$-110-100$

```
000000000000
000000000000 X
000000110 1 X
000000000 0 X
1111 0 0 11 X
0001 1 0 1 X
```

```
0001 0 0 0 0 0 1 0 0   (260)
```

256 $\quad$ 4 $\quad$ 2 $\quad$ 1

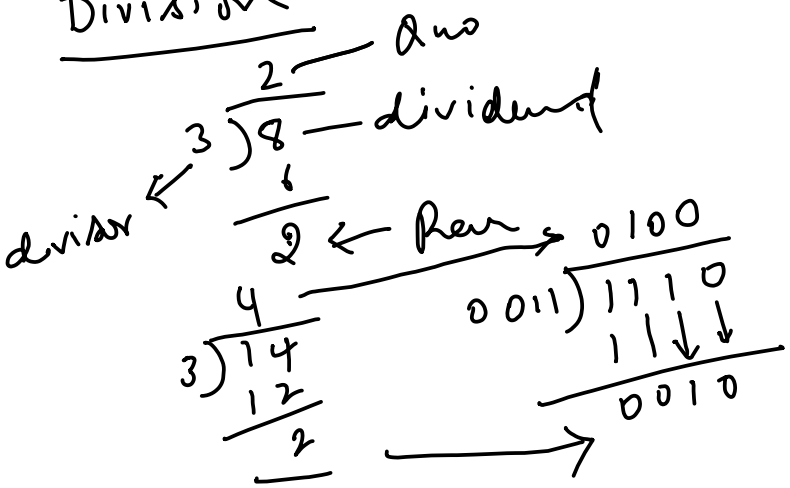
ii) $110011 \times -1 \quad -1 \quad 0$ $\quad\Big|\quad B = 1\,0\,1\,1\,0\,0\,\boxed{0}$

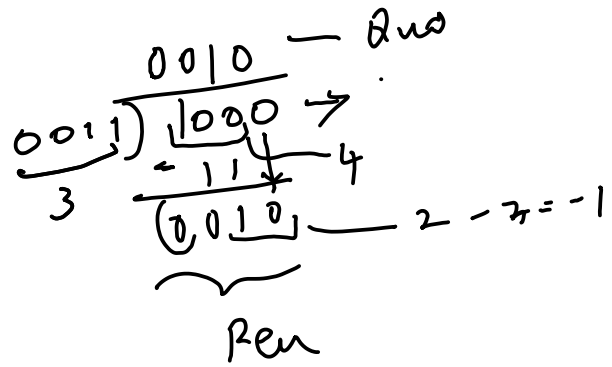
$-1 \quad -1 \quad 0$

```
000000 0 0 0 0 0 0
000000 1 1 0 1 XX
0001 1 0 1 X X
```

```
0 0 0 1 0 0 0 0 1 0 0   (260)
```

Division



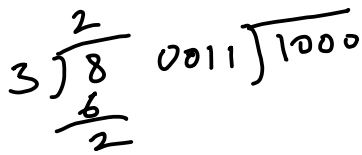


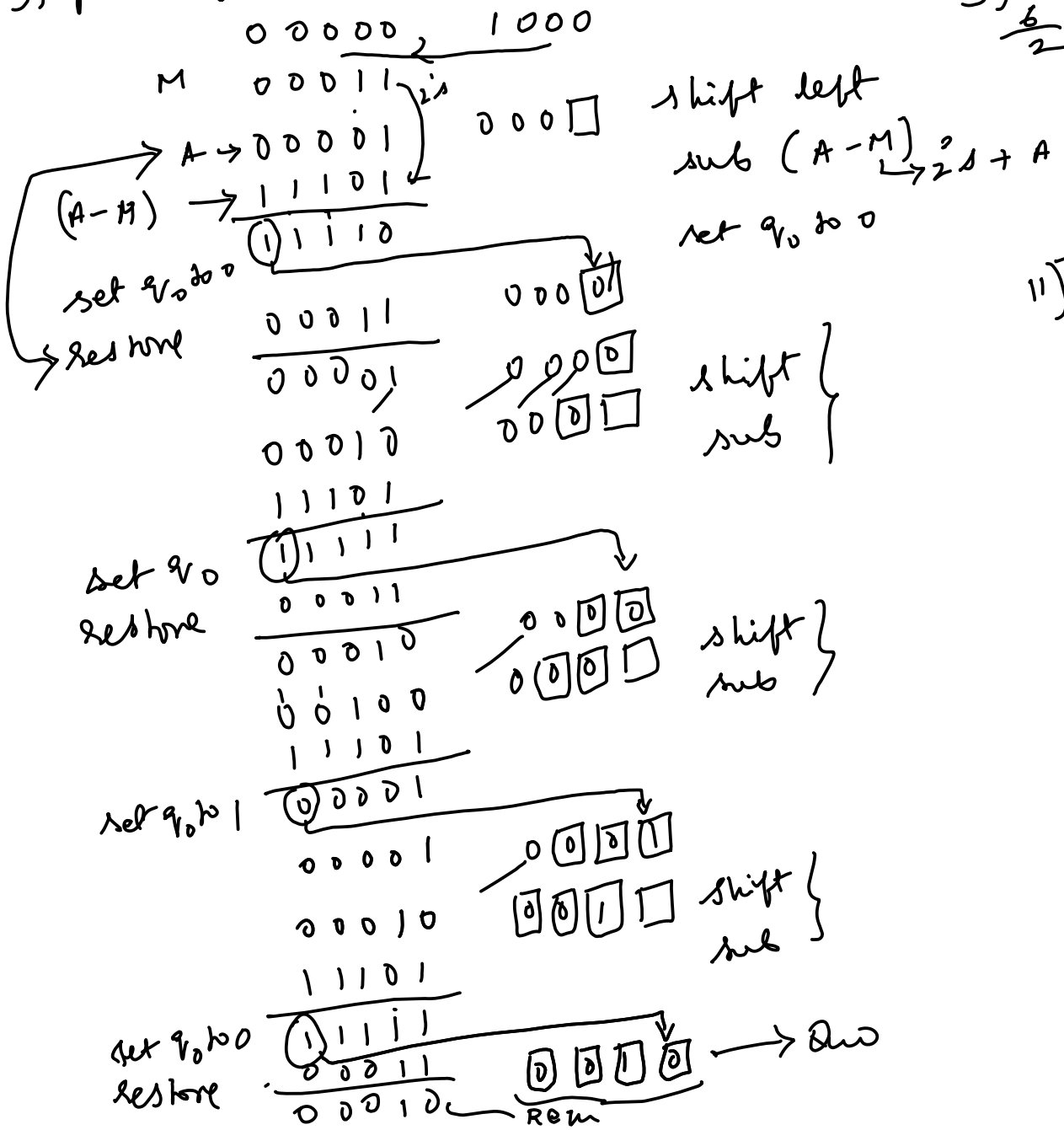
Restoring Division:

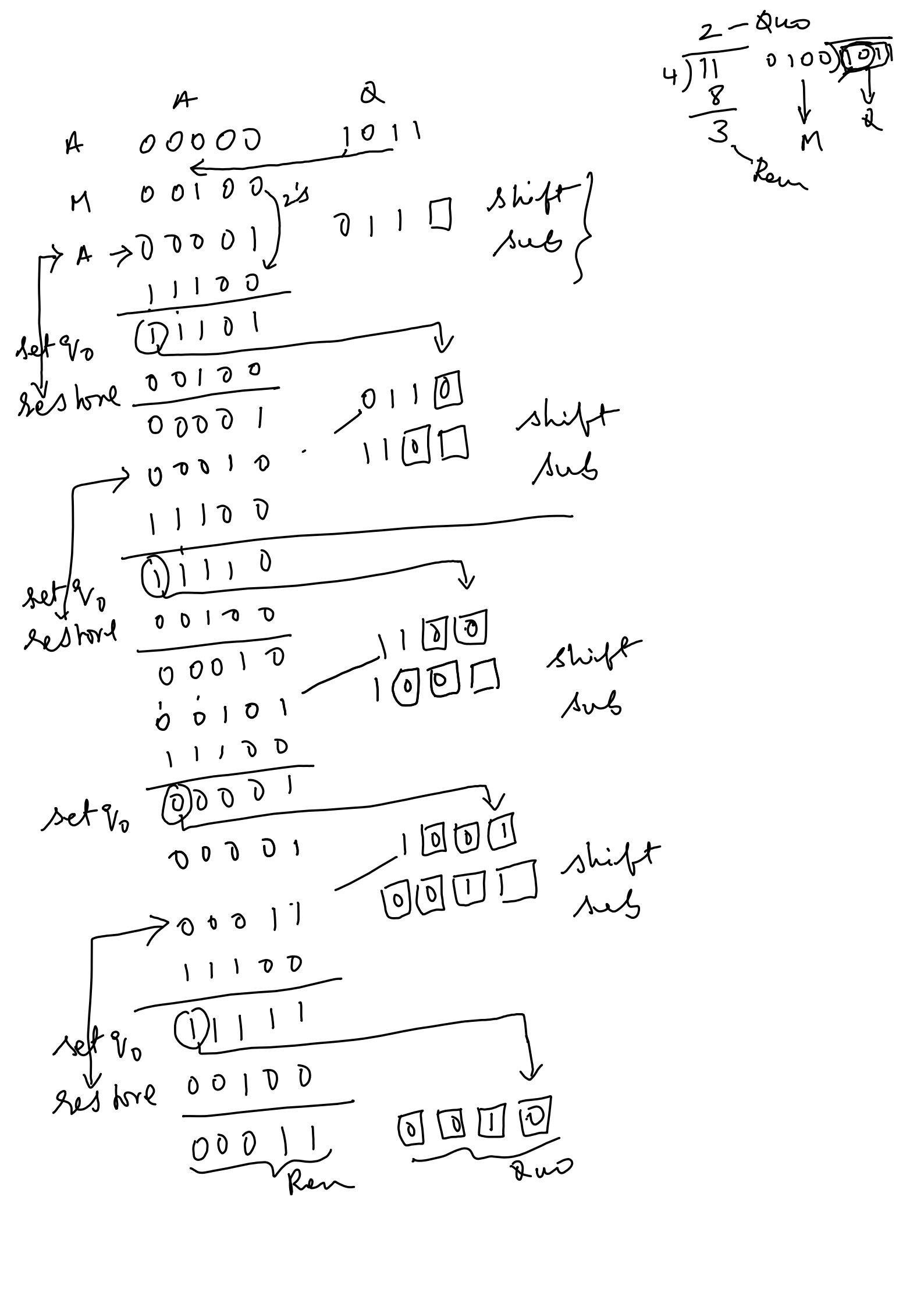
1. Shift A and Q left one binary position
2. Subtract M from A, and place the answer back in A
3. If the sign of A is 1, set q0 to 0 and add M back to A (restore A); otherwise, set q0 to 1
4. Repeat these steps n times

Step 1

Step 2 : If the sign of A is 1, add M back to A





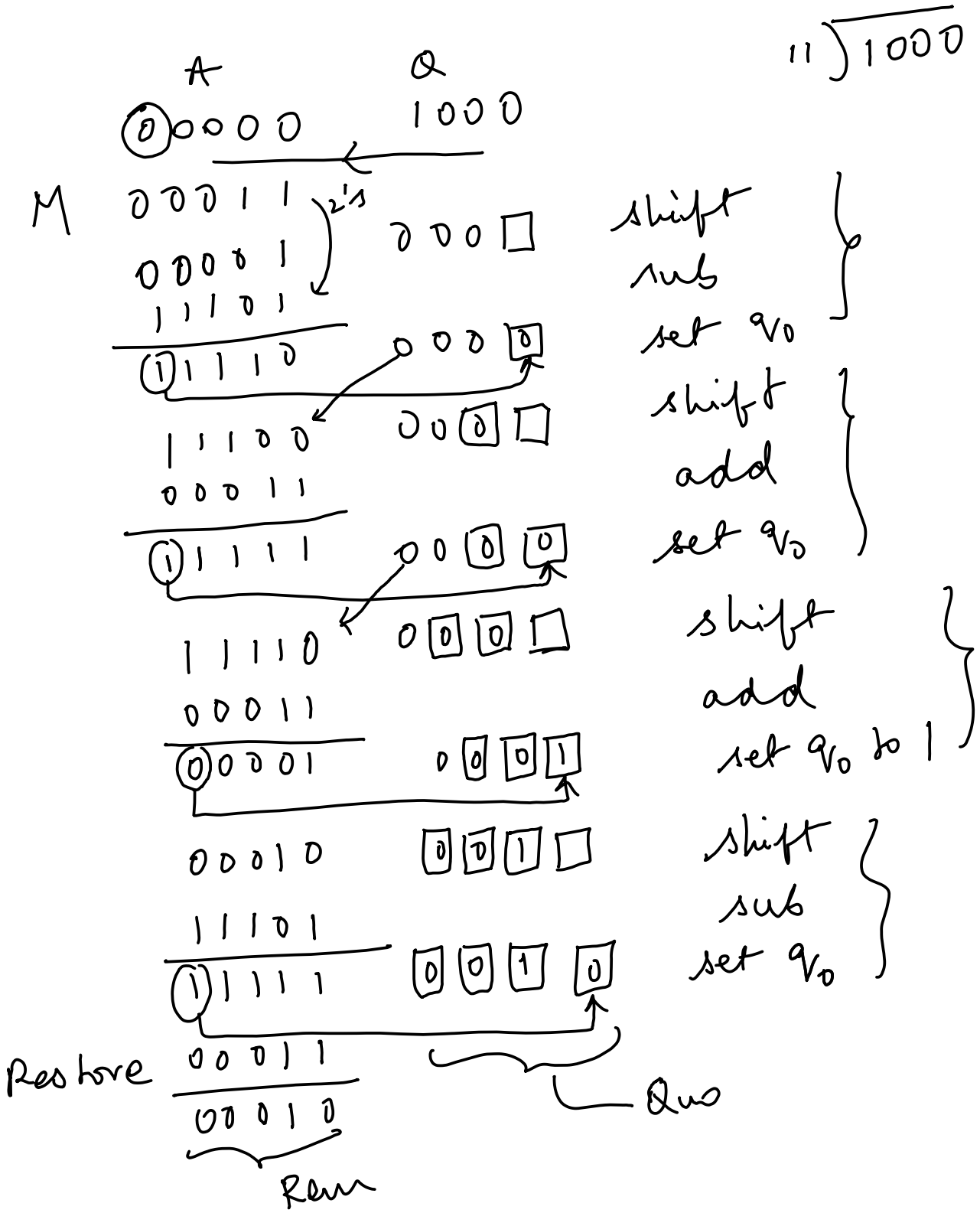

2 - Quo
4)11
8
3
Rem
0100
1011
M
Q
A
A 00000 1011
M 00100
2's
00001
11100
11101
shift
sub
set q0
restore
00100
00001
01110
11101
00010
11100
11110
00100
00010
11010
10001
00101
11100
00001
00001
10001
00111
11100
11111
00100
00011
Rem
0010
Quo

NON-RESTORING DIVISION

Step 1: (Repeat n times)

1. If the sign of A is 0, shift A and Q left one bit position and subtract M from A; otherwise, shift A and Q left and add M to A.
2. Now, if the sign of A is 0, set q0 to 1; otherwise, set q0 to 0.

Step2: If the sign of A is 1, add M to A



Eg: 0.0625_{10} — single / double

$S = 0$, $E = -4$, $M = 0000 - -$

$E^1 = -4 + 127 =$

$E^1 = -4 + 1023 =$

single — $23 + 1 = 24$ bits

double — $52 + 1 = 53$

$1.0000 - -$

1.111×2^{-4}

0.001

$10,000 \times 2^{-1}$

1.0000×2^{1}

$S_A = -1$ $S_B = +$

$-0.0175 \times 10^{2} + 6.8 \times 10^{4}$

$E_A = 2$ $E_B = 4$

A − B
−A − B
A + B
−A + B

$|2 - 4| = 2$

$+0.0175 \times 10^{4}$

$+6.8000 \times 10^{4}$

$+6.8175 \times 10^{4}$

$0.0175 \times 10^{4} + 6.8 \times 10^{4}$

6.8×10^2 + 1.2×10^4

$(6.8 \times 1.2) \times 10^{2+4}$ → $\frac{1.2 \times 10^4}{6.8 \times 10^2}$

$= \left(\frac{1.2}{6.8}\right) \times 10^{4-2}$

A 1.10000×2^{130} — Sing

B 1.01011×2^{140}

$130 = E_A'$ $E_A = 3$

$140 = E_B'$ $E_B = 140 - 127$

$= 13$

( ) × 2

130
140
270

270 − 127 = 143 (E_R')

× 2

2 × 127

$E = 13 + 3 = 16$

$E = 143 - 127 = 16$

$\frac{B}{A} = \frac{1.01011 \times 2^{140}}{1.10000 \times 2^{130}} = 2^{140-130} = 2^{10+127}$

$E_R' = +127$